AI-Based Consulting Proposal Automation

[Pre-Sales Proposal Intelligence Engine/Generative AI for Technical Bid Preparation]

Problem statement

It helps consulting companies and IT service firms that prepare frequent project proposals.
It supports pre-sales teams and solution architects by accelerating proposal drafting.
It reduces manual effort while maintaining structured, professional output.

1️⃣ Business Development / Sales Manager

- Talks to client
- Collects requirement
- Understands budget expectations

2️⃣ Solution Architect

- Designs technical approach
- Chooses architecture and stack
- Estimates effort

This person is usually the key brain in proposal creation.

3️⃣ Technical Lead / Senior Engineer

- Validates feasibility
- Reviews technical decisions
- Checks integration complexity

4️⃣ Project Manager

- Creates timeline
- Plans milestones
- Estimates resource allocation

5️⃣ Finance / Pre-Sales Analyst

- Calculates cost breakdown
- Applies rate cards
- Ensures margin targets

6️⃣ Compliance / Security Consultant (If regulated industry)

- Reviews risk
- Validates regulatory alignment

So normally, generating one proper enterprise proposal involves:

- Cross-functional coordination
- Multiple reviews
- Several hours to days of work

Day 1 – Problem Understanding & Structure Design

What is a consulting proposal made of?

Break proposal into 4 structured components:

1. Technical Approach
2. Timeline
3. Cost Estimation
4. Risk Assessment

Then design structured output format:

```
{
 "technical_approach": "...",
 "timeline": "...",
 "cost_breakdown": "...",
 "risk_assessment": "..."
}
```

Make them design architecture diagram first. No coding.

Day 2 – Local LLM Setup (No Paid API)

Install:

ollama run mistral

Or:

ollama run llama3

Teach:

- Prompt structure
- Role-based prompting
- Controlled output formatting
- JSON output enforcement

Example Prompt Template:

You are a senior consulting architect.

Based on the client requirement below, generate:

1. Technical Approach
2. Timeline
3. Cost Estimation Breakdown
4. Risk Assessment

Client Requirement:
{input}

Return structured output.

Day 3 – Backend API Build

Build FastAPI backend:

Endpoint:

POST /generate-proposal

Input:

```
{
 "client_requirement": "Build Python API for banking system"
}
```

Inside:

- Construct prompt
- Call Ollama
- Return structured response

Teach:

- Separation of logic

- Prompt template file
- Clean architecture

Day 4 – Add Basic Cost Estimation Logic

Add simple rule-based layer:

Example:

- Junior dev rate: $30/hr
- Senior dev rate: $60/hr
- QA rate: $25/hr

Let system:

- Estimate effort (LLM)
- Multiply with rates (Python logic)

This teaches hybrid AI + deterministic logic.

Day 5 – Risk Intelligence Layer

Add structured risk categories:

- Technical risk
- Resource risk
- Timeline risk
- Compliance risk

Have LLM generate risk explanation,
But validate format with schema.

Basic Architecture

Client Requirement
↓
Prompt Builder
↓
Local LLM (Ollama)
↓
Structured JSON Output
↓
Post-processing (Cost Logic)
↓
FastAPI Response

Tech Stack

- FastAPI
- Ollama (local LLM)
- Pydantic
- SQLite (optional for storing proposals)
- Docker (optional if time permits)

Simple Project Structure

```
proposal_generator/
│
├── app/
│   ├── main.py
```

```
│   ├── prompt_builder.py
│   ├── generator.py
│   ├── cost_logic.py
│   ├── schemas.py
│
├── prompts/
│   └── proposal_template.txt
│
├── requirements.txt
└── README.md
```

Learn

- Structured prompting
- Local LLM usage
- API architecture
- Hybrid AI + rule logic
- Output validation
- JSON enforcement
- Controlled generation

—----------

Project Structure

```
proposal_generator/
│
├── app/
│   ├── main.py
```

│   ├── generator.py

│   ├── prompt_builder.py

│   ├── cost_logic.py

│   ├── schemas.py

│

├── requirements.txt

1️⃣ requirements.txt

fastapi

uvicorn

pydantic

requests

Install:

pip install -r requirements.txt

Make sure Ollama is installed and running:

ollama run llama3

(Exit after first run — model downloads once.)

2️⃣ schemas.py

from pydantic import BaseModel

from typing import Dict, List

```
class ProposalRequest(BaseModel):
    project_title: str
    industry: str
    duration_months: int
    expected_users: int
    tech_stack: List[str]


class ProposalResponse(BaseModel):
    executive_summary: str
    technical_approach: str
    timeline: str
    estimated_cost: Dict[str, float]
    risk_assessment: str
```

---

3 prompt_builder.py

```
def build_prompt(data):
    return f"""
You are a senior consulting solution architect.

Generate a structured proposal based on the following:
```

Project Title: {data.project_title}

Industry: {data.industry}

Duration: {data.duration_months} months

Expected Users: {data.expected_users}

Preferred Tech Stack: {', '.join(data.tech_stack)}

Generate:

1. Executive Summary

2. Technical Approach

3. Timeline

4. Risk Assessment

Keep response structured and professional.

"""

---

4 generator.py

```
import requests


OLLAMA_URL = "http://localhost:11434/api/generate"


def generate_proposal(prompt):
    response = requests.post(
        OLLAMA_URL,
```

```
    json={
        "model": "llama3",
        "prompt": prompt,
        "stream": False
    }
)

return response.json()["response"]
```

5️⃣ cost_logic.py

We don’t let AI hallucinate numbers blindly.

```
def calculate_cost(duration_months, expected_users):
    base_dev_cost = 20000 * duration_months
    infra_cost = expected_users * 0.5
    contingency = base_dev_cost * 0.1

    total = base_dev_cost + infra_cost + contingency

    return {
        "development_cost": base_dev_cost,
        "infrastructure_cost": infra_cost,
        "contingency": contingency,
        "total_estimated_cost": total
```

```
}
```

---

6 main.py

```
from fastapi import FastAPI
from app.schemas import ProposalRequest, ProposalResponse
from app.prompt_builder import build_prompt
from app.generator import generate_proposal
from app.cost_logic import calculate_cost

app = FastAPI(title="AI Proposal Generator")


@app.post("/generate-proposal", response_model=ProposalResponse)
def generate(data: ProposalRequest):

  prompt = build_prompt(data)

  llm_output = generate_proposal(prompt)

  cost_data = calculate_cost(
     data.duration_months,
     data.expected_users
  )
```

```
return {
    "executive_summary": llm_output,
    "technical_approach": llm_output,
    "timeline": llm_output,
    "estimated_cost": cost_data,
    "risk_assessment": llm_output
}
```